

# Mobile API Client & Benchmarking Tool

A cross-platform QA utility for API testing, load benchmarking, and response validation — built on React Native and Supabase

| FRONTEND            | BACKEND           | DATA LAYER          | TIMELINE |
|---------------------|-------------------|---------------------|----------|
| React Native (Expo) | Node.js + Express | Supabase (Postgres) | 5 Phases |

Project Overview

This document outlines the technical architecture and phased implementation plan for a mobile-first API client and load-benchmarking tool. The application leverages client-side processing and the device's native network stack to execute and time HTTP requests, while using a managed backend to securely persist request collections, execution history, and user accounts across devices.

The plan is organized into five sequential phases, moving from data foundation and project scaffolding, through the core request engine, into benchmarking and load-testing capability, backend/proxy support with functional assertions, and finally polish and offline resilience.

Core Technology Stack

| Layer            | Technology             | Purpose                                                                                    |
|------------------|------------------------|--------------------------------------------------------------------------------------------|
| Frontend         | React Native (Expo Go) | UI, state management, local hardware execution                                             |
| Backend / Proxy  | Node.js + Express      | Optional proxy for bypassing strict CORS APIs, or serving as a local test-target server    |
| Database & Auth  | Supabase               | Postgres database for storing request collections, test histories, and user authentication |
| State & Fetching | Zustand + Axios/Fetch  | Managing complex UI state and handling HTTP request configurations                         |
| Visualization    | react-native-chart-kit | Rendering benchmark latency distributions and data visualizations                          |

Implementation Roadmap

|    |                                          |
|----|------------------------------------------|
| 01 | Database Schema & Project Initialization |
| 02 | Core Network Engine (The Client)         |
| 03 | Benchmarking & Load Testing Module       |
| 04 | Node.js Proxy & Advanced Assertions      |
| 05 | Polish & Offline Support                 |

1

Database Schema & Project Initialization

Establish a robust data foundation before building any UI

Supabase Schema Design

Before building the UI, establish a relational data foundation in Supabase capable of handling workspaces, folders, and individual API requests.

| Table       | Purpose                                                               |
|-------------|-----------------------------------------------------------------------|
| users       | Managed by Supabase Auth.                                             |
| workspaces  | Top-level containers for projects.                                    |
| collections | Folders used to group related API endpoints.                          |
| requests    | Stores the method (GET, POST), URL, headers (JSON), and body payload. |
| history     | Logs execution results — status code, latency, and timestamp.         |

Expo Initialization

- **Strict typing** — initialize the Expo project with TypeScript for strict type checking on API responses.
- **Navigation** — set up Expo Router for seamless tab navigation between the “Client,” “Collections,” and “Benchmarks” screens.

## 2

## The Core Network Engine

*The heart of the application — profiling the full request lifecycle*

This is the engine of the application. It needs to do more than issue a simple fetch call — it must actively profile the timing and behavior of every request.

### Custom Request Wrapper

- Build a utility function that wraps **fetch** or **axios**.
- **Precise timing** — capture `Date.now()` or `performance.now()` immediately before the request is dispatched and right after the response stream begins, to calculate Time to First Byte (TTFB) and total latency.

### Request Configuration UI

- Implement dynamic forms to input query parameters, headers, and JSON bodies.
- Build an automatic JSON validator that alerts the user if their request body contains syntax errors before sending.

### Response Handling

- Design a collapsible, color-coded JSON viewer for the response payload.
- Display response metadata prominently: status code (color-coded green/red), size (KB), and time (ms).

## 3

## Benchmarking & Load Testing Module

*Realistic mobile load testing under OS network throttling*

Mobile operating systems heavily throttle concurrent network requests. To build a realistic mobile load tester, the architecture must handle batching rather than firing all requests at once.

### The Benchmarking Loop

- Create a module where a user selects a saved request and sets a total iteration count (e.g., 50 requests).
- Implement a controlled asynchronous loop that fires requests in small, sequential batches — preventing Expo Go from crashing the device's network queue.

### Data Aggregation

- Store the latency of each iteration in a temporary array.
- Calculate critical QA metrics: **Min Latency**, **Max Latency**, **Average Latency**, and **Error Rate** (percentage of non-2xx responses).

### Visualizing Performance

- Feed the aggregated data into a line chart or bar graph component to visually represent latency spikes over the duration of the test.

## 4

## Node.js Proxy & Advanced Assertions

*Elevating the tool into a professional QA utility*

To elevate this from a basic tool to a professional QA utility, add functional testing capabilities and optional backend support.

### Node.js Proxy Server (Optional, Recommended)

- Some third-party APIs block requests missing specific browser headers or enforce strict rate limits.
- Build a lightweight Node.js/Express server that accepts a request from the mobile app, forwards it to the target API, and returns the raw response to the phone.

### Functional Assertions

- Add a “Tests” tab to the request configuration screen.
- Allow users to write simple JavaScript assertions — e.g. `expect(response.status).toBe(200)` or `expect(response.body.user).toBeDefined()` — that execute automatically when the response is received.

## 5

## Polish & Offline Support

*Final refinements for a production-ready experience*

### Local Caching

- Implement AsyncStorage or expo-sqlite to cache the latest collections, so the user can browse their API configurations even when offline.

### Syntax Highlighting

- Integrate a library such as react-native-syntax-highlighter to make reading JSON responses and code snippets easy on the eyes.

### Summary

Executed in sequence, these five phases take the project from a well-modeled Supabase schema through a timing-accurate network engine, a batched benchmarking system, optional proxy and assertion tooling, and finally offline resilience and readability polish — resulting in a complete, professional-grade mobile API client and benchmarking tool.